Experiment 9

Student Name: Harish Kumar **UID: 23BCS10168**
Branch: CSE **Section/Group: KRG 2A**
Semester: 6th **Date of Performance: 17/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable Ticket Booking Platform (like BookMyShow) that allows users to discover events, view details, select seats, and book tickets in real-time using a distributed system architecture.

2. **Objective**:
   - To identify functional and non-functional requirements of a ticket booking system
   - To design High Level Architecture (HLD) for booking platforms
   - To understand seat selection and booking mechanisms
   - To design microservices for events, booking, and payments
   - To study seat locking and concurrency control
   - To understand scalability challenges in high-traffic systems

3. **Tools:**
   - Draw.io / Excalidraw – System design diagrams
   - MySQL / PostgreSQL – Booking & transaction data
   - Redis – Caching & seat locking
   - Kafka – Event streaming (booking/payment events)
   - Load Balancer – Traffic distribution
   - CDN – Fast content delivery
   - Microservices Architecture
   - MongoDB – Event data storage

4. **System Requirements:**

A. Functional Requirements:-

- Users should be able to register and login
- Users should search events by title, city, and date
- Users should view event details (description, cast, duration, ratings)
- Users should view seat availability (available/booked)
- Users should select seats and book tickets
- Users should make payments securely
- Users should view booking history
- System should prevent double booking of seats
- System should show booking confirmation in real-time

B. Non-Functional Requirements:-

1) Scalability

a. Support 50–100 million daily active users
b. Handle thousands of concurrent bookings

2) Consistency (CAP Theorem)

a. Strong consistency for seat booking
b. No double booking of same seat

3) Availability

a. Event search and browsing should always be available
b. Slight delay in seat status is acceptable

4) Latency

a. Search response < 200 ms
b. Booking confirmation < 1–2 sec

5) Reliability

a. No loss of booking or payment data
b. System should handle failures gracefully

5. High Level Design (HLD):

System follows Microservices Architecture

Main Components:

1) API Gateway

a. Authentication

b. Authorization

c. Routing

d. Rate Limiting

2) User Service

a. Registration & login

b. User profile management

c. User DB (MySQL)

3) Event Service

a. Stores event details

b. Handles event search

c. Uses MongoDB

4) Search Service

a. Provides fast search results

b. Uses Elasticsearch

5) Booking Service

a. Handles seat selection and booking

b. Coordinates with seat service and payment

6) Seat (Inventory) Service

a. Manages seat availability

b. Tracks booked/available seats

c. Uses PostgreSQL

7) Payment Service

a. Handles payment transactions

b. Integrates with external payment gateway

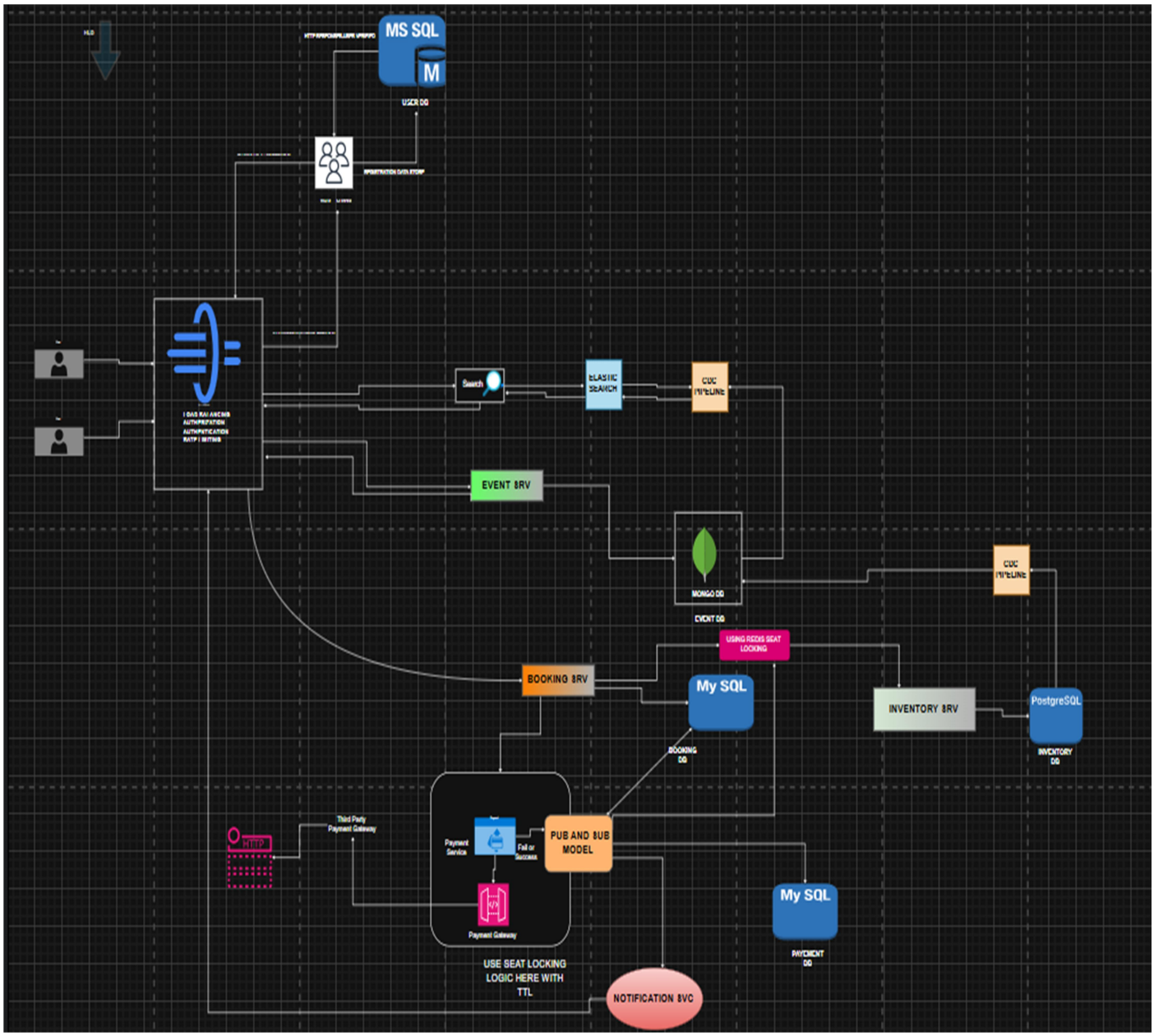
8) Redis (Seat Locking)

a. Temporarily locks seats (TTL)

b. Prevents multiple users from booking same seat

9) Notification Service

a. Sends booking confirmation (SMS/Email)



6. Learning Outcomes (What I Have Learnt):

- Learned how ticket booking platforms are designed
- Understood seat locking and concurrency handling
- Learned booking flow and payment integration
- Understood importance of consistency in booking systems
- Learned microservices-based architecture
- Understood scalability and high availability concepts
- Learned how real-world systems prevent double booking